

Bag of Words for classification

What Bag of Words does

Instead of working with raw text, it converts text into **counts of words**.

Example dataset

```
"I love this movie"  
"This movie is bad"  
"I love this film"
```

Step 1: Build vocabulary

Bag of Words creates a vocabulary like:

```
["bad", "film", "is", "love", "movie", "this"]
```

Step 2: Convert sentences into vectors

Each sentence becomes a **vector of word counts**:

Sentence	bad	film	is	love	movie	this
I love this movie	0	0	0	1	1	1
This movie is bad	1	0	1	0	1	1
I love this film	0	1	0	1	0	0

So text → numbers.

Step 3: Use it in a model

```
from sklearn.feature_extraction.text import CountVectorizer  
from sklearn.linear_model import LogisticRegression  
  
X = [  
    "I love this movie",  
    "This movie is bad",  
    "I love this film"  
]  
  
y = ["positive", "negative", "positive"]
```

Convert text using Bag of Words:

```
vectorizer = CountVectorizer()  
X_bow = vectorizer.fit_transform(X)
```

Encode labels:

```
from sklearn.preprocessing import LabelEncoder  
  
le = LabelEncoder()  
y_encoded = le.fit_transform(y)
```

Train model:

```
model = LogisticRegression()  
model.fit(X_bow, y_encoded)
```

Step 4: Predict new text

```
new_text = ["I love this movie"]  
  
X_new = vectorizer.transform(new_text)  
pred = model.predict(X_new)  
  
le.inverse_transform(pred)
```

Important points

✓ Bag of Words is valid and simple

Good for:

- spam detection
 - sentiment analysis
 - small datasets
-

✗ Limitations

- Ignores word order
(“not good” vs “good” can confuse it)
 - Large vectors for big vocabularies
 - No meaning/semantics
-

Alternatives (better methods)

- TF-IDF (still simple, but better weighting)
 - Word embeddings (Word2Vec, GloVe)
 - Transformers (BERT, GPT embeddings)
-

Final takeaway

- ✓ Yes, you can use Bag of Words
 - ✓ It converts text into word-count vectors
 - ✓ Then you train the model on those vectors
-

Vocabulary

In a basic **Bag of Words (BoW)** model:

- A vocabulary is built during training.
- When a new sentence comes in, the vectorizer checks which vocabulary words are present.
- Words are typically matched as tokens, so "hate" and "hateeee" are considered different words.

Example:

Vocabulary:

```
["hate", "love", "movie"]
```

Sentence:

```
"I hate this movie"
```

Vector:

```
[1, 0, 1]
```

Sentence:

```
"I hateeee this movie"
```

Vector:

```
[0, 0, 1]
```

because "hateeee" is not in the vocabulary.

This leads to the main disadvantage:

BoW understands words, not meaning

Consider these sentences:

```
I like this movie  
I do not like this movie
```

Both contain the word "like".

A simple BoW representation might look very similar for both sentences, even though their meanings are opposite.

Another example:

```
Dog bites man  
Man bites dog
```

BoW sees the same words:

```
dog, bites, man
```

and often produces the same vector, even though the meanings are very different.

So the key limitation is:

Bag of Words captures **word occurrence/counts**, but largely ignores **context, word order, and semantics**.

That's why more advanced approaches were developed:

1. **N-grams** → capture short phrases like "not like".
2. **Word embeddings** → understand that "hate" and "dislike" are similar.
3. **Transformers (BERT, etc.)** → understand context and sentence meaning.

BoW is still useful because it's simple, fast, and often works surprisingly well for basic text-classification tasks such as spam detection or sentiment analysis on smaller datasets.

Get smarter responses, upload files and images, and more.

Prediction

If you cleaned the training data, you **must apply the exact same cleaning to new inputs**.

For example:

```
new_text = "I REALLY LOVE this movie!!!"  
new_text = clean(new_text)  
new_bow = vectorizer.transform([new_text])  
prediction = model.predict(new_bow)
```

The rule is:

```
Training data  
  ↓  
Cleaning  
  ↓  
Vectorizer.fit_transform()  
  ↓  
Train model  
  
New input  
  ↓  
Same Cleaning  
  ↓  
Vectorizer.transform()  
  ↓  
Predict
```

If you clean training data but forget to clean new inputs, the model may perform worse because it's seeing data in a different format than what it learned from.

when predicting:

```
new_text = ["this movie is very good"]  
new_vec = vectorizer.transform(new_text)
```

The vectorizer:

1. Applies the same preprocessing (lowercasing, stop-word removal, etc.).
2. Uses the **same vocabulary learned during training**.
3. Ignores words that were never seen during training.

For example:

Training vocabulary:
['good', 'movie', 'bad', 'film']

New text:
"this movie is very good amazing"

After preprocessing:

movie good amazing

Vector representation:

good	movie	bad	film
1	1	0	0

Notice that "amazing" is ignored because it wasn't in the training vocabulary.

A very important rule:

 Use

```
vectorizer.transform(new_text)
```

for prediction.

 Do **not** use

```
vectorizer.fit_transform(new_text)
```

because that would create a new vocabulary and the model would receive features that don't match what it was trained on.

Author - @Yashas Samaga